دانشکده ریاضی و آمار

زهرا چراغی، محمد ملائی، فاطمه سعیدی

علوم کامپیوتر
نیم‌سال دوم ۰۴-۰۵
هوش مصنوعی

تمرین دوم: مسائل ارضای محدودیت‌ها

۱ حل دستی CSP با Backtracking

سؤال

متغیرها:

$$[A, B, C, D]$$

دامنه‌ها:

$$D(A) = D(B) = D(C) = D(D) = \{1, 2, 3, 4\}$$

قیود:

$$[A \neq B]$$
$$[B < C]$$
$$[A + D = 5]$$
$$[C \neq D]$$
$$[A < C]$$

فرض کنید Backtracking:

- متغیرها را به ترتیب (A, B, C, D) انتخاب می‌کند.
- مقادیر را به ترتیب صعودی امتحان می‌کند.

خواسته‌ها:

الف. درخت جستجو را تا رسیدن به اولین جواب رسم کنید.

ب. تمام نقاطی که Backtrack رخ می‌دهد را مشخص کنید.

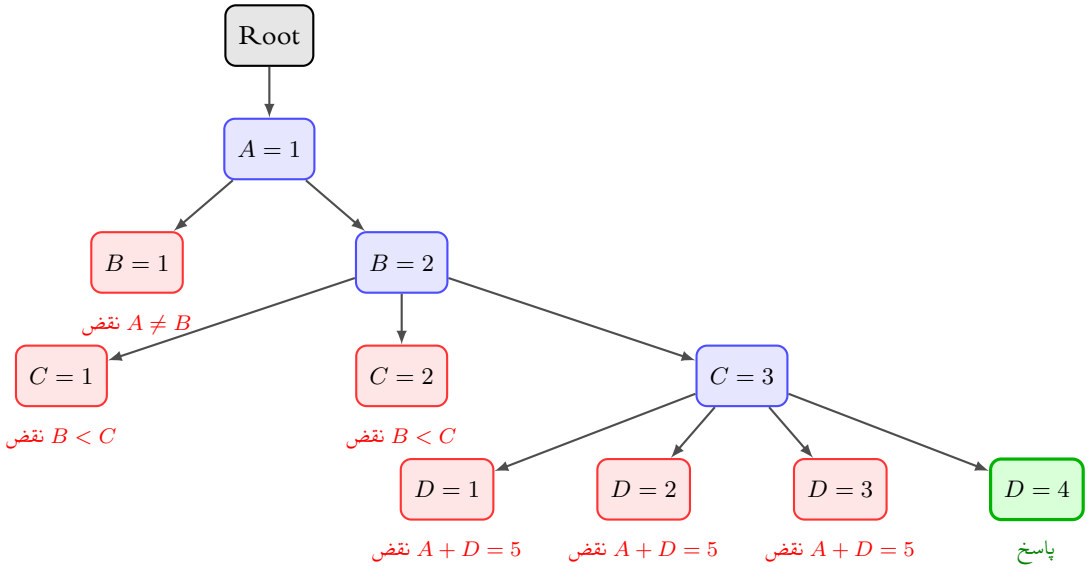
ج. تعداد کل گره‌های تولیدشده را محاسبه کنید.

د. اولین جواب پیدا شده را بنویسید.

ه. اگر به جای ترتیب فعلی، متغیرها با MRV انتخاب شوند، آیا تعداد گره‌های بررسی‌شده کاهش می‌یابد؟ توضیح دهید.

جواب

الف)



ب) مطابق شبه‌کد الگوریتم مطرح شده در کتاب، عقب‌گرد زمانی رخ می‌دهد که پس از بررسی تمام مقادیر یک متغیر هیچ مقدار سازگاری یافت نشود و در نتیجه تابع در انتها دستور return failure را اجرا کرده و به فراخوانی قبلی بازگردد. در این مسئله، هرچند مقادیری مانند $B = 1$، $C = 1$، $C = 2$، $D = 1$، $D = 2$ و $D = 3$ به دلیل برقرار نبودن شرط value is consistent with assignment رد می‌شوند، اما همواره قبل از پایان دامنه، مقدار سازگار بعدی پیدا می‌شود. بنابراین دستور return failure هرگز اجرا نمی‌شود. در نتیجه، تا رسیدن به اولین پاسخ هیچ عقب‌گردی رخ نمی‌دهد.

ج) با احتساب گره ریشه، مجموعاً ۱۱ گره تولید می‌شود:

- گره ریشه: ۱ گره
- بررسی متغیر A: ۱ گره $(A = 1)$
- بررسی متغیر B: ۲ گره $(B = 1, B = 2)$
- بررسی متغیر C: ۳ گره $(C = 1, C = 2, C = 3)$
- بررسی متغیر D: ۴ گره $(D = 1, D = 2, D = 3, D = 4)$

د) اولین انتسابی که تمام قیود را ارضا می‌کند برابر است با:

$$\{A = 1, B = 2, C = 3, D = 4\}$$

هـ) در این قسمت، روش MRV را همراه با Forward Checking در نظر می‌گیریم؛ یعنی پس از هر مقداردهی، دامنهٔ متغیرهای مقدارنگرفته با توجه به قیود مسئله به‌روزرسانی می‌شود.
در ابتدا دامنهٔ همهٔ متغیرها یکسان است:

$$D(A) = D(B) = D(C) = D(D) = \{1, 2, 3, 4\}.$$

بنابراین در گام اول، MRV تفاوتی بین متغیرها تشخیص نمی‌دهد. در حالت تساوی، همان ترتیب اولیه را در نظر می‌گیریم و ابتدا A انتخاب می‌شود. با انتخاب $A = 1$ و اعمال Forward Checking، دامنهٔ متغیرهای باقی‌مانده به صورت زیر تغییر می‌کند:

$$A \neq B \Rightarrow D(B) = \{2, 3, 4\},$$

$$A < C \Rightarrow D(C) = \{2, 3, 4\},$$

$$A + D = 5 \Rightarrow D(D) = \{4\}.$$

اکنون چون متغیر D کمترین تعداد مقدار مجاز باقی‌مانده را دارد، طبق MRV متغیر بعدی D انتخاب می‌شود. با انتخاب $D = 4$ و با توجه به قیود $A < C$ و $C \neq D$ دامنهٔ C برابر می‌شود با $D(C) = \{2, 3\}$. اگر $C = 2$ انتخاب شود، از قید

$B < C$ نتیجه می‌شود $B < 2$ پس تنها مقدار ممکن برای B، مقدار 1 است. اما چون $A = 1$، قید $A \neq B$ نقض می‌شود. بنابراین این شاخه شکست می‌خورد.

در مقدار بعدی: $C = 3$ برای B داریم: $B < C \Rightarrow B \in \{1, 2\}$. اما چون $A = 1$، مقدار $B = 1$ حذف می‌شود و در نتیجه $B = 2$.

پس جواب نهایی به دست می‌آید:

$$\boxed{A = 1, \quad B = 2, \quad C = 3, \quad D = 4}$$

اگر تعداد مقدارهای امتحان‌شده یا بررسی‌های ناسازگار را در نظر بگیریم، MRV همراه با Forward Checking باعث کاهش بررسی‌های اضافی و تعداد گره‌های تولید شده می‌شود.

۲ اعمال MRV و Heuristic Degree

سؤال

یک CSP با متغیرهای زیر در نظر بگیرید:

$$[A, B, C, D, E, F]$$

دامنه‌ها:

Domain	Variable
$\{1, 2\}$	A
$\{1, 2, 3\}$	B
$\{1, 2\}$	C
$\{1, 2, 3, 4\}$	D
$\{1, 2\}$	E
$\{1, 2\}$	F

گراف قیود:

- A با B, C, D در ارتباط است.
- B با A, C, E, F در ارتباط است.
- C با A, B, D, E در ارتباط است.
- D با A, C, F در ارتباط است.
- E با B, C, F در ارتباط است.
- F با B, D, E در ارتباط است.

فرض کنید متغیرهای A و C قبلاً مقداردهی شده‌اند و پس از اعمال Forward Checking دامنه‌ها به شکل زیر درآمده‌اند:

Domain	Variable
$\{2, 3\}$	B
$\{1, 2\}$	D
$\{1\}$	E
$\{1\}$	F

خواسته‌ها:

الف. متغیر بعدی را با MRV انتخاب کنید.

ب. در صورت وجود تساوی، Degree Heuristic را اعمال کنید.

ج. در صورت وجود تساوی مجدد، ترتیب انتخاب را مشخص کنید.

د. فرآیند تصمیم‌گیری را مرحله به مرحله توضیح دهید.

جواب

طبق معیار MRV در کتاب، متغیری باید انتخاب شود که کمترین تعداد مقدار قانونی باقی‌مانده را داشته باشد. بنابراین چون $|D(B)| = 2$، $|D(D)| = 2$، $|D(E)| = 1$ و $|D(F)| = 1$، دو متغیر E و F کمترین دامنه را دارند. پس در این مرحله بین E و F تساوی ایجاد می‌شود.

برای شکستن این تساوی، از Degree Heuristic استفاده می‌کنیم. در این معیار، متغیری انتخاب می‌شود که با تعداد بیشتری از متغیرهای هنوز مقداردهی‌نشده قید داشته باشد. متغیر E با B و F در ارتباط است، پس درجهٔ مؤثر آن برابر 2 است. از طرف دیگر، متغیر F با B، D و E در ارتباط است، پس درجهٔ مؤثر آن برابر 3 می‌شود.بنابراین چون $Degree(F) = 3$ و $Degree(E) = 2$، متغیر F انتخاب می‌شود.

اگر بعد از اعمال MRV و Degree Heuristic باز هم تساوی باقی می‌ماند، می‌توانستیم از ترتیب اولیهٔ متغیرها یعنی $[A, B, C, D, E, F]$ استفاده کنیم؛ اما در این مسئله تساوی شکسته می‌شود.

در نتیجه، متغیر بعدی برای مقداردهی برابر است با F. همچنین چون دامنهٔ آن فقط شامل مقدار 1 است، در صورت ادامهٔ مقداردهی خواهیم داشت: $F = 1$.

۳ اجرای AC-۳

سؤال

متغیرها و دامنه‌ها:

Domain	Variable
$\{1, 2, 3, 4\}$	X
$\{2, 3, 4\}$	Y
$\{1, 2, 3\}$	Z
$\{1, 2\}$	W

قیود:

$$[X < Y]$$
$$[Y < Z]$$
$$[Z \neq W]$$
$$[X \neq W]$$

خواسته‌ها:

الف. صف اولیه AC-3 را تشکیل دهید.

ب. الگوریتم را مرحله به مرحله اجرا کنید.

ج. پس از هر حذف مقدار، صف را به‌روزرسانی کنید.

د. دامنه‌های نهایی را بنویسید.

ه. مشخص کنید آیا مسئله Arc Consistent شده است یا خیر.

و. آیا Arc Consistency به تنهایی جواب مسئله را مشخص می‌کند؟ توضیح دهید.

جواب

الف) در الگوریتم AC-۳، صف اولیه شامل تمامی یال‌های جهت‌دار موجود در گراف محدودیت است. از آنجا که هر محدودیت دودویی بین دو متغیر، دو یال جهت‌دار ایجاد می‌کند، صف اولیه شامل ۸ یال زیر خواهد بود:

$$Queue = \{(X,Y),(Y,X),(Y,Z),(Z,Y),(Z,W),(W,Z),(X,W),(W,X)\}$$

ب ـ ج) در جدول زیر فرآیند خروج یال‌ها از صف، بررسی سازگاری محدودیت‌ها و به‌روزرسانی صف نشان داده شده است:

مرحله	یال خروجی	بررسی شرط	تغییرات در دامنه (حذف مقادیر ناسازگار)	یال‌های افزوده	صف به‌روزرسانی‌شده (باقی‌مانده)
۱	(X,Y)	$X < Y$	عدد 4 از D_X حذف می‌شود. $\rightarrow D_X = \{1,2,3\}$	(W,X)	$(Y,X),(Z,Y),(Z,W),(W,Z),(X,W),(W,X)$
۲	(Y,X)	$Y > X$	دامنه‌ها سازگارند. تغییری نمی‌کند.	—	$(Y,Z),(Z,Y),(Z,W),(W,Z),(X,W),(W,X)$
۳	(Y,Z)	$Y < Z$	اعداد 3 و 4 از D_Y حذف می‌شوند. $\rightarrow D_Y = \{2\}$	(X,Y)	$(Z,Y),(Z,W),(W,Z),(X,W),(W,X),(X,Y)$
۴	(Z,Y)	$Z > Y$	اعداد 1 و 2 از D_Z حذف می‌شوند. $\rightarrow D_Z = \{3\}$	(W,Z)	$(Z,W),(W,Z),(X,W),(W,X),(X,Y)$
۵	(Z,W)	$Z \neq W$	دامنه‌ها سازگارند. تغییری نمی‌کند.	—	$(W,Z),(X,W),(W,X),(X,Y)$
۶	(W,Z)	$W \neq Z$	دامنه‌ها سازگارند. تغییری نمی‌کند.	—	$(X,W),(W,X),(X,Y)$
۷	(X,W)	$X \neq W$	دامنه‌ها سازگارند. تغییری نمی‌کند.	—	$(W,X),(X,Y)$
۸	(W,X)	$W \neq X$	دامنه‌ها سازگارند. تغییری نمی‌کند.	—	(X,Y)
۹	(X,Y)	$X < Y$	اعداد 2 و 3 از D_X حذف می‌شوند. $\rightarrow D_X = \{1\}$	(W,X)	(W,X)
۱۰	(W,X)	$W \neq X$	عدد 1 از D_W حذف می‌شود. $\rightarrow D_W = \{2\}$	(Z,W)	(Z,W)
۱۱	(Z,W)	$Z \neq W$	دامنه‌ها سازگارند. تغییری نمی‌کند.	—	صف خالی می‌شود. پایان الگوریتم.

د) پس از اتمام کامل الگوریتم AC-۳، دامنه‌های نهایی متغیرها به این شکل کاهش می‌یابند:

- $D_X = \{1\}$
- $D_Y = \{2\}$
- $D_Z = \{3\}$
- $D_W = \{2\}$

ه) بله. مسئله به طور کامل Consistent Arc شده است؛ چرا که صف الگوریتم با موفقیت خالی شده و هیچ‌یک از دامنه‌ها تهی نشده‌اند.

و) در این مسئله خاص، بله. در حالت کلی، الگوریتم سازگاری یال تنها دامنه‌ها را هرس می‌کند و لزوماً به یک پاسخ قطعی منجر نمی‌شود و نیاز به جستجوی عقب‌گرد دارد؛ اما در این مسئله، دامنه هر متغیر دقیقاً به یک مقدار واحد کاهش یافته است. از آنجا که هیچ متغیری بیش از یک انتخاب ندارد و هیچ دامنه‌ای هم تهی نشده است، الگوریتم توانسته فضای جستجو را کاملاً حل کند و جواب یکتا را به صورت زیر مشخص سازد:

$$X = 1, \quad Y = 2, \quad Z = 3, \quad W = 2$$

۴ سؤال ۴: برنامه‌نویسی زمان‌بندی امتحانات

صورت سؤال

یک دانشگاه دارای ۶ درس زیر است:

Database	AI
Networks	Operating Systems
Algorithms	Compiler

چهار بازه زمانی برای برگزاری امتحان‌ها وجود دارد:

$$[\mathrm{T}_1, \mathrm{T}_2, \mathrm{T}_3, \mathrm{T}_4]$$

زوج درس‌هایی که دانشجوی مشترک دارند و بنابراین نباید هم‌زمان برگزار شوند عبارت‌اند از:

درس اول	درس دوم
AI	Database
AI	Algorithms
Database	Operating Systems
Database	Networks
Operating Systems	Networks
Networks	Compiler
Compiler	Algorithms

خواسته‌ها:

الف. مسئله را به صورت CSP مدل‌سازی کنید.

ب. Backtracking ساده را پیاده‌سازی کنید.

ج. نسخه دارای MRV را پیاده‌سازی کنید.

د. نسخه دارای MRV + Degree Heuristic را پیاده‌سازی کنید.

ه. تعداد گره‌های تولیدشده در هر سه روش را گزارش کنید.

و. تمام زمان‌بندی‌های معتبر را پیدا کنید.

ز. بهترین زمان‌بندی را از نظر کمینه کردن تعداد امتحان‌ها در بازه T_4 پیدا کنید.

جواب

در این بخش مسئله زمان‌بندی امتحانات به صورت یک مسئله ارضای قید (CSP) مدل‌سازی و با سه روش زیر حل شده است: Backtracking ساده، Backtracking + MRV و Backtracking + MRV + Degree Heuristic. کد مربوط به این بخش در فایل `../exam_scheduling_csp.py` قرار دارد.

مدل‌سازی مسئله به صورت **CSP**

در این مسئله، هر درس یک متغیر و هر بازه زمانی یک مقدار ممکن برای آن متغیر است. بنابراین:

$$X = \{\mathrm{AI}, \mathrm{Database}, \mathrm{Operating\ Systems}, \mathrm{Networks}, \mathrm{Compiler}, \mathrm{Algorithms}\}$$

دامنه همه متغیرها یکسان است:

$$D(X_i) = \{\mathrm{T}_1, \mathrm{T}_2, \mathrm{T}_3, \mathrm{T}_4\}$$

قیود مسئله از نوع دودویی و نامساوی هستند. اگر دو درس دانشجوی مشترک داشته باشند، نباید در یک بازه زمانی برگزار شوند. بنابراین قید کلی به صورت زیر است:

$$X_i \neq X_j$$

جفت‌های دارای تداخل عبارت‌اند از:

درس دوم	درس اول
Database	AI
Algorithms	AI
Operating Systems	Database
Networks	Database
Networks	Operating Systems
Compiler	Networks
Algorithms	Compiler

این مدل دقیقاً مشابه مسئله رنگ‌آمیزی گراف است؛ درس‌ها رأس‌های گراف، بازه‌های زمانی رنگ‌ها، و تداخل‌ها یال‌های گراف هستند.

توضیح کوتاه پیاده‌سازی

در کد، لیست درس‌ها در متغیر `courses` ، بازه‌های زمانی در `time_slots` و قیود در `constraints` ذخیره شده‌اند. سپس از روی قیود، دیکشنری همسایگی `adjacency` ساخته شده است تا بتوان برای هر درس، درس‌های متداخل با آن را سریع پیدا کرد.

تابع `is_consistent` بررسی می‌کند که آیا مقداردهی یک درس با مقداردهی‌های قبلی همسایه‌های آن سازگار است یا خیر. تابع `get_legal_values` نیز مقدارهای مجاز فعلی یک متغیر را با توجه به انتساب‌های انجام‌شده برمی‌گرداند.

تابع اصلی جستجو `backtracking_solver` یک تابع عمومی است که استراتژی انتخاب متغیر را به عنوان ورودی می‌گیرد. در هر مرحله، متغیر بعدی انتخاب می‌شود و مقدارها با ترتیب ثابت T_1, T_2, T_3, T_4 امتحان می‌شوند. این تابع با پارامتر `find_all` می‌تواند یا پس از یافتن اولین جواب متوقف شود، یا کل فضای جستجو را برای یافتن همه زمان‌بندی‌های معتبر پیمایش کند.

روش **Backtracking** ساده

در روش ساده، ترتیب انتخاب متغیرها ثابت و برابر با ترتیب زیر است:

[AI, Database, Operating Systems, Networks, Compiler, Algorithms]

در هر گام، اولین درس مقداردهی‌نشده انتخاب می‌شود. مقدارها نیز همیشه به ترتیب T_1 تا T_4 امتحان می‌شوند. اگر مقدار امتحان‌شده با همسایه‌های قبلاً مقداردهی‌شده ناسازگار باشد، همان مقدار رد می‌شود.

روش **MRV**

در روش MRV یا Minimum Remaining Values در هر گام متغیری انتخاب می‌شود که کمترین تعداد مقدار قانونی باقی‌مانده را داشته باشد. یک مقدار قانونی است اگر با مقداردهی همسایه‌های قبلاً مقداردهی‌شده تناقض نداشته باشد.

در صورت تساوی بین چند متغیر، ترتیب اولیه درس‌ها برای شکستن تساوی استفاده می‌شود. در این سؤال از Least Constraining Value استفاده نشده است و ترتیب مقدارها همچنان ثابت است.

روش **MRV + Degree Heuristic**

در روش سوم، ابتدا همان معیار MRV اعمال می‌شود. اگر چند متغیر کمترین تعداد مقدار قانونی را داشته باشند، Degree Heuristic برای شکستن تساوی به کار می‌رود.

در این معیار، متغیری انتخاب می‌شود که بیشترین تعداد قید با متغیرهای هنوز مقداردهی‌نشده داشته باشد؛ بنابراین هنگام محاسبه درجه، فقط همسایه‌هایی شمرده می‌شوند که هنوز مقدار نگرفته‌اند. اگر پس از این مرحله نیز تساوی باقی بماند، ترتیب اولیه درس‌ها معیار نهایی است.

نحوه شمارش گره‌های تولیدشده

طبق تعریف صورت سؤال، گره ریشه یعنی انتساب خالی به عنوان یک گره شمرده شده است. همچنین هر بار که الگوریتم یک مقدار را برای یک متغیر امتحان می‌کند، یک گره جدید تولید می‌شود؛ حتی اگر آن مقدار بلافاصله ناسازگار باشد و رد شود.

نتایج عددی

در اجرای اصلی برنامه، ابتدا هر سه روش فقط تا رسیدن به اولین جواب اجرا شدند. جدول زیر تعداد گره‌های تولیدشده تا اولین جواب را نشان می‌دهد:

روش	تعداد گره‌های تولیدشده
Backtracking ساده	11
Backtracking + MRV	11
Backtracking + MRV + Degree Heuristic	13

در این نمونه، روش MRV تا اولین جواب تعداد گره‌ها را نسبت به روش ساده کاهش نداده است، زیرا در بسیاری از مراحل تساوی رخ می‌دهد و با شکستن تساوی بر اساس ترتیب اولیه، همان ترتیب روش ساده به دست می‌آید. اما افزودن Degree Heuristic ترتیب انتخاب متغیرها را تغییر می‌دهد و در این مسیر خاص اولین جواب با 13 گره به دست می‌آید.
برای تولید همه جواب‌ها، برنامه یک بار فضای کامل را با روش Backtracking ساده پیمایش کرد. تعداد کل زمان‌بندی‌های معتبر برابر با ۴۸۰ است و در این پیمایش کامل 1365 گره تولید شد.

همه جواب‌ها

تمام ۴۸۰ زمان‌بندی معتبر در فایل متنی زیر ذخیره شده‌اند:

`exam_scheduling_solutions.txt`

این فایل شامل خلاصه مدل، تعداد گره‌های تولیدشده، همه جواب‌ها، و همچنین جواب‌های بهینه از نظر کمینه کردن تعداد امتحان‌های بازه T_4 است.
یک نمونه زمان‌بندی معتبر که توسط روش ساده به عنوان اولین جواب پیدا شد:

درس	بازه زمانی
AI	T_1
Database	T_2
Operating Systems	T_1
Networks	T_3
Compiler	T_1
Algorithms	T_2

بهترین جواب‌ها بر اساس کمینه کردن تعداد امتحان‌های T_4

برای هر جواب، تعداد درس‌هایی که در بازه T_4 قرار گرفته‌اند شمرده شد:

$$\text{cost}(S) = |\{X_i \mid S(X_i) = \text{T}_4\}|$$

کمترین مقدار ممکن برابر با ۰ است؛ یعنی می‌توان تمام امتحان‌ها را فقط با سه بازه T_1,T_2,T_3 زمان‌بندی کرد و نیازی به استفاده از T_4 نیست. تعداد جواب‌هایی که این کمینه را دارند برابر با ۳۰ است.
چند نمونه از جواب‌های بهینه:

شماره	AI	Database	OS	Networks	Compiler	Algorithms
1	T_1	T_2	T_1	T_3	T_1	T_2
2	T_1	T_2	T_1	T_3	T_1	T_3
3	T_1	T_2	T_1	T_3	T_2	T_3

لیست کامل ۳۰ جواب بهینه نیز در همان فایل `exam_scheduling_solutions.txt` در بخش Best schedules آمده است.

۵ سؤال ۵: سودوکو

صورت سؤال

سودوکوی ۶×۶ زیر را در نظر بگیرید. بلوک‌ها ۳ ستون در ۲ سطر هستند.

–	6	–	–	–	1
–	–	–	3	–	–
2	–	–	–	5	–
–	–	4	–	–	–
–	–	2	–	–	–
5	–	–	–	2	–

خواسته‌ها:

الف. مسئله را به صورت CSP مدل‌سازی کنید.

ب. Backtracking ساده را پیاده‌سازی کنید.

ج. MRV را اضافه کنید.

د. Forward Checking را اضافه کنید.

ه. AC-3 را به عنوان مرحله پیش‌پردازش اضافه کنید.

و. تعداد گره‌های تولیدشده در چهار حالت زیر را مقایسه کنید: Backtracking، Backtracking + MRV، Backtrack-ing + MRV + Forward Checking و AC-3 + MRV + Forward Checking.

ز. نتایج را در قالب یک جدول مقایسه‌ای گزارش کنید.

جواب

کد مربوط به این بخش در فایل `../sudoku_csp.py` قرار دارد. تابع `main` به صورت خلاصه جدول مقایسه، جواب نهایی، و خلاصه اثر AC-3 را چاپ می‌کند.

مدل‌سازی **CSP**

یک مسئله ارضای قید (CSP) از سه مؤلفه تشکیل می‌شود: متغیرها، دامنه‌ها و قیدها. در این بخش، سودوکوی ۶×۶ با بلوک‌های ۳×۲ (۳ ستون و ۲ سطر) را به صورت CSP مدل می‌کنیم.

*

متغیرها

هر خانه از صفحه یک متغیر است. با استفاده از نمایش (row-major):

$$X = \{X_{r,c} \mid r \in \{1,\dots,6\},\ c \in \{1,\dots,6\}\} \quad \Rightarrow \quad |X| = 36$$

*

دامنه‌ها

- برای خانه‌های از پیش پر شده: $D(X_{r,c}) = \{v\}$ که v همان مقدار داده‌شده است.
- برای خانه‌های خالی: $D(X_{r,c}) = \{1,2,3,4,5,6\}$

در این مسئله ۹ خانه از پیش پر شده و ۲۷ خانه خالی وجود دارد. مقادیر از پیش داده‌شده عبارت‌اند از:

$$D(X_{1,1}) = \{1\}, \quad D(X_{1,5}) = \{6\}, \quad D(X_{2,3}) = \{3\}, \quad D(X_{3,2}) = \{5\}, \quad D(X_{3,6}) = \{2\},$$
$$D(X_{4,4}) = \{4\}, \quad D(X_{5,4}) = \{2\}, \quad D(X_{6,2}) = \{2\}, \quad D(X_{6,6}) = \{5\}.$$

*

قیدها

قیدهای سودوکو از نوع همه متفاوت (all-different) هستند و به صورت مجموعه‌ای از قیدهای دودویی نامساوی پیاده‌سازی می‌شوند. دو خانه اگر «همسایه» باشند (یعنی در یک سطر، یک ستون یا یک بلوک مشترک قرار داشته باشند) باید مقادیر متفاوت داشته باشند:

$$X_i \neq X_j \qquad \forall\, (i,j) \in \text{Peers}$$

سه نوع قید وجود دارد:

۱. قید سطری: در هر سطر، همه ۶ مقدار متمایز باشند.

$$\forall r,\ \forall c_1 \neq c_2 :\ X_{r,c_1} \neq X_{r,c_2}$$

۲. قید ستونی: در هر ستون، همه ۶ مقدار متمایز باشند.

$$\forall c,\ \forall r_1 \neq r_2 :\ X_{r_1,c} \neq X_{r_2,c}$$

۳. قید بلوک: در هر بلوک ۳×۲، همه ۶ مقدار متمایز باشند. صفحه به ۶ بلوک تقسیم می‌شود (دو ردیف بلوک × سه ستون بلوک):

بلوک ۰ (سطرهای ۱–۲، ستون‌های ۱–۳)	بلوک ۱ (سطرهای ۱–۲، ستون‌های ۴–۶)
بلوک ۲ (سطرهای ۳–۴، ستون‌های ۱–۳)	بلوک ۳ (سطرهای ۳–۴، ستون‌های ۴–۶)
بلوک ۴ (سطرهای ۵–۶، ستون‌های ۱–۳)	بلوک ۵ (سطرهای ۵–۶، ستون‌های ۴–۶)

تعداد کل قیدهای دودویی:

$$6 \times \binom{6}{2}\ \text{(سطری)}\ +\ 6 \times \binom{6}{2}\ \text{(ستونی)}\ +\ 6 \times \binom{6}{2}\ \text{(بلوکی)}\ -\ \text{همپوشانی}\ =\ 216$$

هر خانه دقیقاً ۱۲ همسایه دارد (۵ در سطر + ۵ در ستون + ۲ در بلوک که قبلاً نبوده‌اند).

سودوکوی اولیه

جدول زیر وضعیت اولیه سودوکو را نشان می‌دهد. خانه‌های رنگی مقادیر از پیش داده‌شده‌اند.

–	6	–	–	–	1
–	–	–	3	–	–
2	–	–	–	5	–
–	–	4	–	–	–
–	–	2	–	–	–
5	–	–	–	2	–

روش‌های حل

Backtracking ساده

در این روش، متغیرها به ترتیب ثابت سطر–ستون انتخاب می‌شوند:

$$X_{1,1},\ X_{1,2},\ \ldots,\ X_{1,6},\ X_{2,1},\ \ldots,\ X_{6,6}$$

خانه‌های از پیش پر شده از همان ابتدا در مقداردهی اولیه قرار دارند و رد می‌شوند. مقادیر به ترتیب صعودی $1,2,3,4,5,6$ آزمایش می‌شوند. در هر گام، سازگاری مقدار انتخابی با تمام همسایه‌های مقداردهی‌شده بررسی می‌شود. اگر هیچ مقداری مناسب نبود، الگوریتم به گام قبل برمی‌گردد.

شمارش گره: هر بار که یک مقدار برای یک متغیر آزمایش می‌شود یک گره شمرده می‌شود، حتی اگر آن مقدار فوری ناسازگار باشد.

Backtracking با MRV

اکتشافی MRV (Minimum Remaining Values) در هر گام متغیری را انتخاب می‌کند که کمترین تعداد مقدار قانونی باقی‌مانده را داشته باشد. یک مقدار در صورتی «قانونی» است که با تمام همسایه‌های مقداردهی‌شده سازگار باشد:

$$\text{LegalValues}(X_{r,c}) = \{v \in D(X_{r,c}) \mid \forall\, nb \in \text{Peers}(X_{r,c}) \text{ assigned},\ \text{assignment}[nb] \neq v\}$$

در صورت تساوی، متغیری که زودتر در ترتیب سطر–ستون قرار دارد انتخاب می‌شود. دامنه‌ها در این روش تغییر نمی‌کنند؛ فیلتر به صورت پویا و در لحظه انتخاب انجام می‌شود.

Backtracking با MRV و Forward Checking

Forward Checking پس از هر مقداردهی $X = v$، مقدار v را از دامنه همه همسایه‌های هنوز مقداردهی‌نشده حذف می‌کند:

$$\forall\, nb \in \text{Peers}(X) \text{ (unassigned)}: \quad D(nb) \leftarrow D(nb) \setminus \{v\}$$

اگر دامنه هر همسایه‌ای خالی شد، مقداردهی فعلی رد و الگوریتم بازمی‌گردد. این روش شکست‌های قریب‌الوقوع را زودتر تشخیص می‌دهد.

پیاده‌سازی از کپی دامنه‌ها در هر سطح بازگشت استفاده می‌کند (هر فراخوانی بازگشتی یک نسخه جدید از دامنه‌ها می‌گیرد) تا بازگشت به حالت قبل به صورت خودکار انجام شود.

AC-3 به عنوان پیش‌پردازش

AC-3 یک الگوریتم سازگاری قوسی است که پیش از شروع جستجو اجرا می‌شود و دامنه‌ها را تا حد ممکن کوچک می‌کند.

الگوریتم **AC-3** مطابق شبه‌کد کتاب:

Algorithm 1 AC-3(csp)

```
queue ← all directed arcs in csp
while queue is not empty do
    (X_i, X_j) ← POP(queue)
    if REVISE(csp, X_i, X_j) then
        if |D_i| = 0 then
            return False
        for each X_k ∈ Neighbors(X_i) \ {X_j} do
            add (X_k, X_i) to queue
return True
```

الگوریتم **REVISE** مطابق شبه‌کد کتاب:

Algorithm 2 REVISE(csp, X_i, X_j)

```
revised ← False
for each x ∈ D_i do
    if there is no y ∈ D_j such that (x, y) satisfies the constraint between X_i and X_j then
        delete x from D_i
        revised ← True
return revised
```

پس از اجرای AC-3 یا همه یال‌ها سازگار قوسی هستند، یا دامنه یک متغیر تهی شده و مشخص می‌شود که مسئله با دامنه‌های فعلی قابل حل نیست. در سودوکو، قید بین دو خانه همسایه از نوع نامساوی است؛ بنابراین در تابع REVISE مقدار x از D_i فقط وقتی حذف می‌شود که در D_j هیچ مقدار متفاوتی برای پشتیبانی از آن وجود نداشته باشد. در حالت خاص قید $X_i \neq X_j$، این دقیقاً وقتی رخ می‌دهد که $D_j = \{x\}$ باشد.

تأثیر **AC-3** روی این سودوکو:

AC-3 پیش از جستجو ۱۰۶ مقدار از دامنه‌های ۲۷ خانه خالی حذف کرد (از ۱۶۲ مقدار به ۵۶ مقدار). مهم‌تر از آن، ۹ خانه اضافه به دامنه تک‌عضوی رسیدند (تمام ۶ خانه خالی سطر اول و ۶ خانه خالی سطر دوم کاملاً مشخص شدند):

خانه	مقدار تعیین‌شده توسط **AC-3**
$X_{1,2}$	4
$X_{1,3}$	2
$X_{1,4}$	5
$X_{1,6}$	3
$X_{2,1}$	5
$X_{2,2}$	6
$X_{2,4}$	1
$X_{2,5}$	2
$X_{2,6}$	4

دامنه‌های پس از AC-3 برای تمام خانه‌ها (سطرهای ۳ تا ۶):

{3}	{6}	{5}	{2}	{4}	{1}
{4}	{2}	{1}	{3}	{6}	{5}
{2}	{1, 3}	{3, 6}	{1, 4, 6}	{5}	{3, 4, 6}
{1, 6}	{1, 3, 5}	{4}	{1, 6}	{1, 3}	{2, 3, 6}
{1, 6}	{1, 3, 4}	{2}	{1, 4, 5, 6}	{1, 3}	{3, 4, 6}
{5}	{1, 3, 4}	{3, 6}	{1, 4, 6}	{2}	{3, 4, 6}

خانه‌های آبی: مقادیر اولیه یا مشخص‌شده توسط AC-3

جدول مقایسه‌ای

جدول زیر تعداد گره‌های تولیدشده در هر روش را نشان می‌دهد. گره به هر بار آزمایش یک مقدار برای یک متغیر اطلاق می‌شود، حتی اگر آن آزمایش فوری شکست بخورد یا توسط Forward Checking رد شود.

روش	تعداد گره‌های تولیدشده	وضعیت حل
Backtracking ساده	204	حل شد
MRV + Backtracking	144	حل شد
Forward Checking + MRV + Backtracking	35	حل شد
Forward Checking + MRV + AC-3	35	حل شد

جواب نهایی سودوکو

هر چهار روش به جواب یکسانی رسیدند:

3	6	5	2	4	1
4	2	1	3	6	5
2	3	6	1	5	4
1	5	4	6	3	2
6	4	2	5	1	3
5	1	3	4	2	6

آبی: داده‌های اولیه سبز: مقادیر حل‌شده